INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	INTRODUCCION AL DESARROLLO DE NUEVAS PLATAFORMAS (E)				
TÍTULO DE LA PRÁCTICA:	Plano interactivo de un ambiente con Canvas en Jetpack Compose				
NÚMERO DE LA PRÁCTICA:	07	**AÑO LECTIVO:**	2026	**NRO. SEMESTRE:**	B
FECHA DE PRESENTACIÓN:	15/10/2026	**HORA DE PRESENTACIÓN:**	11:59:00		
INTEGRANTE(s): - Hancco Mullisaca, Sergio Danilo - Huacani Jara, Denise Andrea				**NOTA (0 - 20):**	Nota colocada por el docente
DOCENTE: Roxana Evelyn Limache Calatayud					

SOLUCIÓN Y RESULTADOS

I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS

Ejercicio 1: Plano Interactivo en Jetpack Compose

Enunciado

Desarrollar una aplicación interactiva en Jetpack Compose denominada **PlanoInteractivo_Compose**, en la cual se represente un plano arquitectónico simple con salones y marcadores interactivos dibujados mediante la API de gráficos de Compose (`Canvas`).

Desarrollo Paso a Paso

Paso 1: Creación del Proyecto

Se creó un nuevo proyecto en Android Studio seleccionando la plantilla **Empty Activity (Compose)**. Se configuró el nombre del proyecto como `PlanoInteractivo_Compose`, el paquete base `com.example.planointeractivo_compose`, lenguaje Kotlin y el SDK mínimo compatible con API 34 o superior.

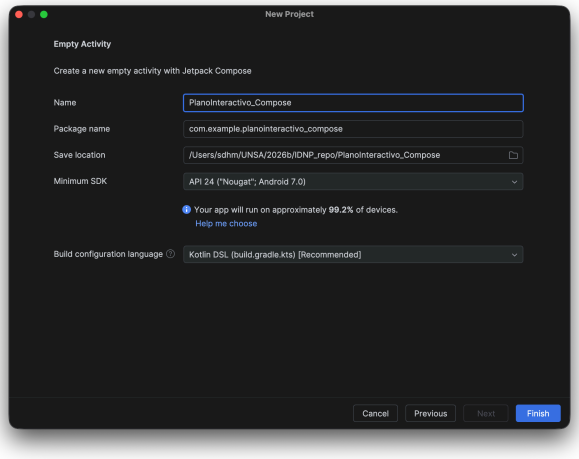


Figura 1: Configuración inicial del nuevo proyecto en Android Studio.

Paso 2: Configuración de Dependencias e Importaciones

En el archivo `MainActivity.kt`, debajo de la definición del paquete, se incorporaron las importaciones necesarias para el soporte de gráficos en Compose (`Canvas`, `Offset`, `Size`, `Color`), manejo de eventos táctiles (`pointerInput`, `detectTapGestures`), administración de estado reactivo (`remember`, `mutableStateOf`, `getValue`, `setValue`) y componentes visuales de Material 3 (`AlertDialog`, `Surface`, `Text`, `TextButton`).

```
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.Canvas
import androidx.compose.foundation.gestures.detectTapGestures
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.material3.AlertDialog
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.geometry.Offset
import androidx.compose.ui.geometry.Size
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.input.pointer.pointerInput
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import com.example.planointeractivo_compose.ui.theme.PlanoInteractivo_ComposeTheme
```

Figura 2: Declaración de importaciones requeridas en MainActivity.kt.

Paso 3: Definición del Modelo de Datos (`Marcador`)

Fuera de cualquier función composable, se definió la clase de datos (`data class`) `Marcador`. Esta estructura encapsula las propiedades geométricas y descriptivas de cada punto de interés: las coordenadas en el plano (`offset: Offset`), el radio de cobertura táctil (`radio: Float`), así como el título y la descripción textual del ambiente.

```
data class Marcador(
    val offset: Offset,
    val radio: Float,
    val titulo: String,
    val descripcion: String
)
```

Figura 3: Definición de la data class Marcador.

Paso 4: Inicialización del Composable `PlanoAmbiente` y Gestión de Estado

Se implementó la función composable `PlanoAmbiente`, inicializando una lista inmutable de marcadores de muestra mediante `remember` (Sala A y Sala B con sus respectivas coordenadas cartesianas y radios de 40f). Asimismo, se declaró la variable reactiva `marcadorSeleccionado` delegada por `mutableStateOf<Marcador?>(null)` para controlar la selección activa y la visibilidad del diálogo.

```
@Composable
fun PlanoAmbiente() {
    val marcadores = remember {
        listOf(
            Marcador(Offset( x = 100f,  y = 150f),  radio = 40f,  titulo = "Sala A",  descripcion = "Sala de cómputo 1"),
            Marcador(Offset( x = 250f,  y = 150f),  radio = 40f,  titulo = "Sala B",  descripcion = "Sala de cómputo 2")
        )
    }
    var marcadorSeleccionado by remember { mutableStateOf<Marcador?>( value = null) }
// el Canvas se agrega en el Paso 5
}
```

Figura 4: Declaración del Composable PlanoAmbiente y variables de estado iniciales.

Paso 5: Declaración del Lienzo `Canvas` y Reconocedor de Gestos

Se declaró el componente `Canvas` con un modificador que ocupa todo el ancho disponible (`fillMaxWidth()`) y una altura fija de `300.dp`. Se vinculó el detector de gestos táctiles mediante `Modifier.pointerInput(Unit)` y la función `detectTapGestures`, preparando el bloque de captura de toques antes de definir el renderizado gráfico.

```
    Canvas(
        modifier = Modifier
            .fillMaxWidth()
            .height(300.dp)
            .pointerInput( key1 = Unit) {
                detectTapGestures { offset ->
                // aquí se calcula el marcador tocado, en el Paso 8
                }
            }
    ) {
    // aquí se dibuja el plano, en los Pasos 6 y 7
    }
```

Figura 5: Configuración del Canvas con modificador pointerInput y detectTapGestures.

Paso 6: Renderizado de la Estructura Arquitectónica (Salones y Pared)

Dentro del bloque de dibujo del `Canvas`, se renderizaron las áreas estructurales del plano. Utilizando `drawRect`, se dibujaron el **Salón 1** (color azul claro `0xFFBBDEFB`) y el **Salón 2** (color verde claro `0xFFC8E6C9`) a partir de sus desplazamientos iniciales y dimensiones. Posteriormente, se trazó la pared divisoria vertical con `drawLine` en color gris oscuro (`Color.DarkGray`) con un grosor de trazo de `4f`.

```
Canvas(
    modifier = Modifier
        .fillMaxWidth()
        .height(300.dp)
        .pointerInput( key1 = Unit) {
            detectTapGestures { offset ->
                // Salón 1
                drawRect(
                    color = Color(0xFFBBDEFB),
                    topLeft = Offset( x = 20f,  y = 20f),
                    size = Size( width = 150f,  height = 250f)
                )
                // Salón 2
                drawRect(
                    color = Color(0xFFC8E6C9),
                    topLeft = Offset( x = 180f,  y = 20f),
                    size = Size( width = 150f,  height = 250f)
                )
                // Pared divisoria
                drawLine(
                    color = Color.DarkGray,
                    start = Offset( x = 170f,  y = 20f),
                    end = Offset( x = 170f,  y = 270f),
                    strokeWidth = 4f
                )
                // aquí se dibujan los marcadores, en el Paso 7
            }
        }
```

Figura 6: Renderizado geométrico de los salones y el muro divisorio con drawRect y drawLine.

Paso 7: Renderizado de Marcadores Interactivos

Se iteró sobre la colección `marcadores` dentro del lienzo utilizando `drawCircle`. Cada marcador se dibujó en color rojo en la posición correspondiente (`marcador.offset`) y con el radio predeterminado (`marcador.radio`), completando el renderizado visual de los puntos de interés sobre el plano arquitectónico.

```
        strokeWidth = 4f
    )
    // Pared divisoria
    drawLine(
        color = Color.DarkGray,
        start = Offset( x = 170f,  y = 20f),
        end = Offset( x = 170f,  y = 270f),
        strokeWidth = 4f
    )
    marcadores.forEach { marcador ->
        drawCircle(
            color = Color.Red,
            radius = marcador.radio,
            center = marcador.offset
        )
    }
}
```

Figura 7: Dibujo de los marcadores circulares sobre el plano mediante drawCircle.

Paso 8: Detección de Colisiones Táctiles

En el manejador `detectTapGestures`, se implementó el cálculo euclidiano para verificar si la pulsación táctil intersecta con algún marcador. Al recibir la coordenada `offset` del toque, se evalúa si la distancia `(offset - marcador.offset).getDistance()` es menor o igual al radio del elemento. En caso afirmativo, se actualiza el estado `marcadorSeleccionado`.

```
Canvas(
    modifier = Modifier
        .fillMaxWidth()
        .height(300.dp)
        .pointerInput( key1 = Unit) {
            detectTapGestures { offset ->
                marcadores.forEach { marcador ->
                    if ((offset - marcador.offset).getDistance() <= marcador.radio) {
                        marcadorSeleccionado = marcador
                    }
                }
            }
        }
){
                // Salón 1
```

Figura 8: Lógica matemática de colisión y selección en detectTapGestures.

Paso 9: Integración del Diálogo de Información (`AlertDialog`)

Fuera del bloque `Canvas`, se utilizó la función de alcance `marcadorSeleccionado?.let { ... }` para condicionar el despliegue del componente `AlertDialog`. El diálogo muestra dinámicamente el título y la descripción del marcador pulsado, e incorpora un botón de confirmación («Cerrar») y la acción `onDismissRequest` que restablecen el estado a `null` para descartar la ventana modal.

```
2 Usages
@Composable
fun PlanoAmbiente() {

    // Lista de marcadores del plano
    val marcadores = remember {
        listOf(
            Marcador(
                Offset( x = 100f, y = 150f),
                radio = 40f,
                titulo = "Sala A",
                descripcion = "Sala de cómputo 1"
            ),
            Marcador(
                Offset( x = 250f, y = 150f),
                radio = 40f,
                titulo = "Sala B",
                descripcion = "Sala de cómputo 2"
            )
        )
    }

    // Estado del marcador seleccionado
    var marcadorSeleccionado by remember {
        mutableStateOf<Marcador?>( value = null)
    }

    // Lienzo interactivo
    Canvas(
        modifier = Modifier
            .fillMaxWidth()
            .height(300.dp)
            .pointerInput( key1 = marcadores) {
                detectTapGestures { offset ->
```

Figura 9: Incorporación del diálogo modal AlertDialog para la visualización de detalles.

Paso 10: Integración en `MainActivity` y Vista Previa

En la clase `MainActivity`, dentro del método `onCreate` y el bloque `setContent`, se invocó `PlanoAmbiente` envuelto en el tema de la aplicación (`PlanoInteractivo_ComposeTheme`) y un contenedor `Surface`. Adicionalmente, se configuró una función de previsualización anotada con `@Preview` (`PlanoAmbientePreview`) para verificar el diseño en tiempo de desarrollo.

```
1 Usage
class MainActivity : ComponentActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        setContent {
            PlanoInteractivo_ComposeTheme {
                Surface(
                    modifier = Modifier.fillMaxSize()
                ) {
                    PlanoAmbiente()
                }
            }
        }
    }
}
```

Figura 10: Invocación del Composable en setContent de MainActivity y función Preview.

Paso 11: Ejecución y Validación en Emulador

Se ejecutó la aplicación en el emulador de Android (API 34). Se constató el renderizado fiel de los ambientes arquitectónicos y marcadores, así como la respuesta interactiva al pulsar sobre cada uno de ellos, abriendo correctamente los diálogos modales informativos de «Sala A» y «Sala B».

Pruebas de Ejecución - Estudiante 1

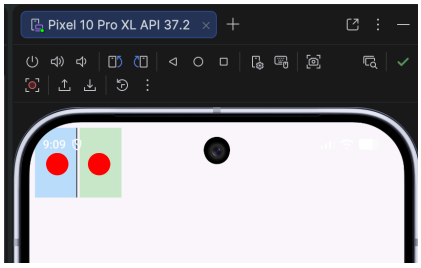


Figura 11: Visualización del plano y marcadores (Estudiante 1).

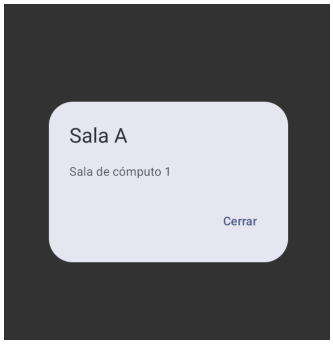


Figura 12: Diálogo modal: Sala A (Estudiante 1).



Figura 13: Diálogo modal: Sala B (Estudiante 1).

Pruebas de Ejecución - Estudiante 2

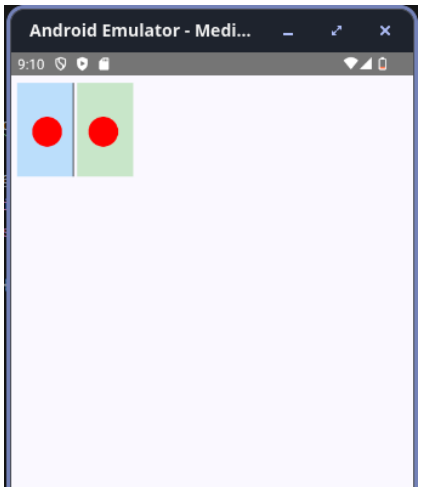


Figura 14: Visualización del plano y marcadores (Estudiante 2).



Figura 15: Diálogo modal: Sala A (Estudiante 2).



Figura 16: Diálogo modal: Sala B (Estudiante 2).

Ejercicio 2: Ampliación del Plano Arquitectónico Interactivo

Enunciado

Amplía el plano resuelto por el docente para representar la visualización interior de al menos una edificación del campus:

1. Amplía la lista de marcadores y agrega los rectángulos/líneas necesarios para representar al menos 4 ambientes (salones y/o patios) de una edificación real del campus, cada uno con su etiqueta visible en el plano.
2. Convierte la lista de marcadores en datos cargados desde un archivo (por ejemplo un .json o .csv en assets/) en lugar de estar escritos directamente en el código, siguiendo un patrón de persistencia similar al usado en el Laboratorio 03.
3. Traslada el estado del plano y sus marcadores a un ViewModel (PlanoViewModel), siguiendo el patrón de ViewModel visto en el Laboratorio 05, en lugar de mantenerlo directamente en el Composable.

4. Integra este plano como una nueva pantalla accesible desde MapaScreen (Laboratorio 05) o desde una tarjeta del Laboratorio 06.

Entregable: informe con capturas del plano con al menos 4 ambientes, del diálogo de información al pulsar cada marcador, y una breve descripción de cómo se cargan los datos desde el archivo.

Desarrollo Paso a Paso

Propuesta 1

Paso 1: Amplía la lista de marcadores y agrega los rectángulos/líneas necesarios para representar al menos 4 ambientes (salones y/o patios) de una edificación real del campus, cada uno con su etiqueta visible en el plano.
Se representó la distribución del **Pabellón de Ingeniería de Sistemas (UNSA)** mediante la clase `Ambiente` (`topLeft`, `size`, `color`, `etiqueta`), cubriendo 6 ambientes reales: Aula A101, Aula A102, Lab. Cómputo, Patio Central, Biblioteca y Cafetería, cada uno con un `Marcador` interactivo asociado.

```
data class Marcador(val offset: Offset, val radio: Float, val titulo: String, val descripcion: String)
data class Ambiente(val topLeft: Offset, val size: Size, val color: Color, val etiqueta: String)
```

En `PlanoAmbiente` se dibujaron el contorno, ambientes y muros con `drawRect` y `drawLine`. Las etiquetas textuales se trazaron con `drawIntoCanvas` y `nativeCanvas.drawText`, mientras que los marcadores circulares se dibujaron con `drawCircle`.

```
// Renderizado de ambientes y etiquetas
ambientes.forEach { amb ->
    drawRect(color = amb.color, topLeft = amb.topLeft, size = amb.size)
    drawRect(color = Color(0xFF90A4AE), topLeft = amb.topLeft, size = amb.size, style = Stroke(3f))
    drawIntoCanvas { canvas ->
        val paint = Paint().apply { color = android.graphics.Color.BLACK; textSize = 20f; textAlign = Paint.Align.CENTER; isFakeBoldText = true }
        canvas.nativeCanvas.drawText(amb.etiqueta, amb.topLeft.x + amb.size.width / 2f, amb.topLeft.y + amb.size.height - 12f, paint)
    }
}
// Marcadores circulares
marcadores.forEach { m ->
    drawCircle(color = Color(0xFFE53935), radius = m.radio, center = m.offset)
    drawCircle(color = Color.White, radius = m.radio, center = m.offset, style = Stroke(4f))
}
```

La detección de toques mediante `detectTapGestures` calcula la distancia euclidiana hacia cada marcador para actualizar `marcadorSeleccionado` y abrir un `AlertDialog` informativo.



Figura 17: Visualización general del plano.



Figura 18: Diálogo informativo: Aula A101.



Figura 19: Diálogo informativo: Lab. Cómputo.

Paso 2: Convierte la lista de marcadores en datos cargados desde un archivo (por ejemplo un .json o .csv en assets/) en lugar de estar escritos directamente en el código, siguiendo un patrón de persistencia similar al usado en el Laboratorio 03.
Se desacopló la lista fija de marcadores creando el archivo de datos `app/src/main/assets/marcadores.csv`, estructurado en formato tabular con cabecera conceptual `x,y,radio,titulo,descripcion`:

```
100,115,20,Aula A101,Salón de clases del primer nivel
272,115,20,Aula A102,Salón de clases del primer nivel
100,245,20,Lab. Cómputo,Laboratorio de cómputo
272,245,20,Patio Central,Patio de recreo y descanso
100,360,20,Biblioteca,Sala de lectura y estudio
272,360,20,Cafetería,Servicio de alimentos
```

Siguiendo el patrón de lectura línea a línea del Laboratorio 03, se implementó la función `cargarMarcadores(context: Context)` que lee el archivo mediante `context.assets.open(...).bufferedReader()`. Se utiliza `lineSequence()` para omitir líneas vacías o comentarios, separando los campos por comas (`split(",")`) e instanciando cada `Marcador`. En el Composable `PlanoAmbiente` se reemplazó la lista estática por `remember { cargarMarcadores(context) }`.

```
fun cargarMarcadores(context: Context): List<Marcador> {
    return try {
        val contenido = context.assets.open("marcadores.csv").bufferedReader().use
{ it.readText() }
        contenido.lineSequence()
            .map { it.trim() }
            .filter { it.isNotEmpty() && !it.startsWith("#") }
            .mapNotNull { linea ->
```

```
                val partes = linea.split(",")
                if (partes.size >= 5) {
                    Marcador(
                        offset = Offset(partes[0].trim().toFloat(), partes[1].trim().toFloat()),
                        radio = partes[2].trim().toFloat(),
                        titulo = partes[3].trim(),
                        descripcion = partes.drop(4).joinToString(",").trim()
                    )
                } else null
            }.toList()
    } catch (e: Exception) { emptyList() }
}
```

```
# marcadores.csv -> x,y,radio,titulo,descripcion
100,115,20,Aula A101,Salón de clases del primer nivel
272,115,20,Aula A102,Salón de clases del primer nivel
100,245,20,Lab. Cómputo,Laboratorio de cómputo
272,245,20,Patio Central,Patio de recreo y descanso
100,360,20,Biblioteca,Sala de lectura y estudio
272,360,20,Cafetería,Servicio de alimentos
```

Figura 20: Archivo assets/marcadores.csv en Android Studio.



Figura 21: Plano cargando marcadores desde el CSV.



Figura 22: Diálogo informativo cargado dinámicamente.

Paso 3: Traslada el estado del plano y sus marcadores a un ViewModel (PlanoViewModel), siguiendo el patrón de ViewModel visto en el Laboratorio 05, en lugar de mantenerlo directamente en el Composable.

Siguiendo el patrón arquitectónico del Laboratorio 05, se incorporaron las dependencias de ciclo de vida en `libs.versions.toml` y `app/build.gradle.kts`:

```
lifecycleViewmodel = "2.11.0"
androidx-lifecycle-viewmodel-ktx = { group = "androidx.lifecycle", name = "lifecycle-viewmodel-ktx", version.ref = "lifecycleViewmodel" }
androidx-lifecycle-viewmodel-compose = { group = "androidx.lifecycle", name = "lifecycle-viewmodel-compose", version.ref = "lifecycleViewmodel" }
```

Se creó la clase `PlanoViewModel : AndroidViewModel(application)` dentro del paquete `viewmodel/`, centralizando el estado mutable fuera de la composición mediante propiedades observables con encapsulación (`private set`), y delegando la carga desde `assets/marcadores.csv` en su bloque `init`:

```kotlin
class PlanoViewModel(application: Application) : AndroidViewModel(application) {
    var marcadores by mutableStateOf<List<Marcador>>(emptyList())
        private set
    var marcadorSeleccionado by mutableStateOf<Marcador?>(null)
        private set

    init { marcadores = cargarMarcadores(application) }

    fun seleccionarMarcador(marcador: Marcador?) { marcadorSeleccionado = marcador }
    fun limpiarSeleccion() { marcadorSeleccionado = null }
}
```

En `MainActivity.kt`, se desacopló la UI mediante elevación de estado (*state hoisting*): `PlanoAmbiente` inyecta `PlanoViewModel = viewModel()` y delega el renderizado a un Composable puro `PlanoContenido` que recibe el estado y expone lambdas para los eventos, permitiendo previsualizaciones directas con `@Preview` sin depender de la instancia del ViewModel.

```kotlin
@Composable
fun PlanoAmbiente(viewModel: PlanoViewModel = viewModel()) {
    PlanoContenido(
        marcadores = viewModel.marcadores,
        marcadorSeleccionado = viewModel.marcadorSeleccionado,
        onMarcadorClick = { m -> viewModel.seleccionarMarcador(m) },
        onCerrar = { viewModel.limpiarSeleccion() }
    )
}
```

La compilación y validación se ejecutó exitosamente mediante `./gradlew :app:compileDebugKotlin`.

Paso 4: Integra este plano como una nueva pantalla accesible desde MapaScreen (Laboratorio 05) o desde una tarjeta del Laboratorio 06.

Se configuró el soporte de navegación y recursos iconográficos extendidos en `libs.versions.toml` y `app/build.gradle.kts`:

```
navigationCompose = "2.9.5"
androidx-navigation-compose = { group = "androidx.navigation", name = "navigation-compose",
version.ref = "navigationCompose" }
androidx-compose-material-icons-extended = { group = "androidx.compose.material", name =
"material-icons-extended" }
```

En `MainActivity.kt`, se definieron los destinos mediante una clase sellada `Pantalla` y se implementó el contenedor `NavHost` dentro del Composable raíz `PlanoApp`:

```kotlin
sealed class Pantalla(val route: String) {
    object Mapa : Pantalla("mapa")
    object Plano : Pantalla("plano")
}

@Composable
fun PlanoApp() {
    val navController = rememberNavController()
    NavHost(navController = navController, startDestination = Pantalla.Mapa.route) {
        composable(Pantalla.Mapa.route) {
            MapaScreen(onAbrirPlano = { navController.navigate(Pantalla.Plano.route) })
        }
        composable(Pantalla.Plano.route) {
```

```
            PlanoScreen(onVolver = { navController.popBackStack() })
        }
    }
}
```

Se diseñó la pantalla `MapaScreen` incorporando una tarjeta estilizada (`Card` con esquinas redondeadas de `20.dp`, ícono de ubicación y flecha indicadora, siguiendo el patrón de diseño de tarjetas del Laboratorio 06). Al pulsar sobre ella, se ejecuta la acción de navegación hacia `PlanoScreen`, la cual incluye una barra superior con botón de retorno (`popBackStack()`) y renderiza el plano interactivo administrado por el `PlanoViewModel`:

```
// Tarjeta interactiva en MapaScreen (estilo Laboratorio 06)
Card(
    modifier = Modifier.fillMaxWidth().clip(RoundedCornerShape(20.dp)).clickable(onClick = onAbrirPlano),
    colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.secondaryContainer)
) {
    Row(modifier = Modifier.fillMaxWidth().padding(16.dp), verticalAlignment = Alignment.CenterVertically) {
        Box(modifier = Modifier.size(48.dp).clip(CircleShape).background(MaterialTheme.colorScheme.primary), contentAlignment = Alignment.Center) {
            Icon(Icons.Default.Place, contentDescription = null, tint = MaterialTheme.colorScheme.onPrimary)
        }
        Column(modifier = Modifier.weight(1f).padding(horizontal = 16.dp)) {
            Text("Pabellón de Ingeniería", style = MaterialTheme.typography.titleMedium, fontWeight = FontWeight.Bold)
            Text("Plano interactivo con 6 ambientes del campus", style = MaterialTheme.typography.bodySmall)
        }
        Icon(Icons.AutoMirrored.Filled.ArrowForward, contentDescription = null)
    }
}
```



Figura 23: Pantalla MapaScreen con la tarjeta del pabellón.



Figura 24: Transición y visualización de PlanoScreen.

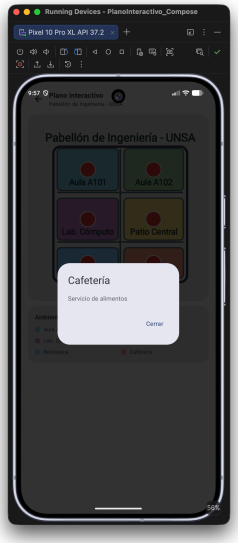


Figura 25: Interacción táctil: Diálogo Aula A102.



Figura 26: Interacción táctil: Diálogo Cafetería.

Propuesta 2

Paso 1: Amplía la lista de marcadores y agrega los rectángulos/líneas necesarios para representar al menos 4 ambientes (salones y/o patios) de una edificación real del campus, cada uno con su etiqueta visible en el plano.

Se representa el interior de una edificación con 5 ambientes: Laboratorio 1, Laboratorio 2, Aula 301, Sala de Docentes y Patio Central.

- Se agrega la data class `Ambiente` (posición, tamaño, color y etiqueta) para dibujar los rectángulos. `Marcador` se mantiene igual.

- El `Canvas` usa `aspectRatio` en lugar de una altura fija, para conservar las proporciones del plano.
- Las etiquetas se dibujan con `drawText` y un `TextMeasurer` (`rememberTextMeasurer()`).
- Se dibuja el contorno del edificio con `Stroke` y las paredes internas con `drawLine`.

```
data class Ambiente(
    val topLeft: Offset,
    val size: Size,
    val color: Color,
    val etiqueta: String
)

@Composable
fun PlanoAmbiente() {
    val ambientes = remember {
        listOf(
            Ambiente(Offset(10f, 10f), Size(170f, 150f), Color(0xFFBBDEFB), "Laboratorio 1"),
            Ambiente(Offset(180f, 10f), Size(170f, 150f), Color(0xFFC8E6C9), "Laboratorio 2"),
            Ambiente(Offset(10f, 160f), Size(170f, 120f), Color(0xFFFFE0B2), "Aula 301"),
            Ambiente(Offset(180f, 160f), Size(170f, 120f), Color(0xFFE1BEE7), "Sala de
Docentes"),
            Ambiente(Offset(10f, 280f), Size(340f, 110f), Color(0xFFFFF9C4), "Patio Central")
        )
    }
    val marcadores = remember {
        listOf(
            Marcador(Offset(95f, 85f), 16f, "Laboratorio 1", ""),
            Marcador(Offset(265f, 85f), 16f, "Laboratorio 2", ""),
            Marcador(Offset(95f, 220f), 16f, "Aula 301", ""),
            Marcador(Offset(265f, 220f), 16f, "Sala de Docentes", ""),
            Marcador(Offset(180f, 335f), 16f, "Patio Central", "")
        )
    }

    ...

        // Ambientes con su etiqueta
        ambientes.forEach { ambiente ->
            drawRect(
                color = ambiente.color,
                topLeft = ambiente.topLeft * escala,
                size = ambiente.size * escala
            )
            drawText(
                textMeasurer = medidorTexto,
                text = ambiente.etiqueta,
                topLeft = ambiente.topLeft * escala + Offset(10f, 8f),
                style = TextStyle(fontSize = 12.sp, color = Color.Black)
            )
        }
    }
```

Resultado

Se ven los 5 ambientes con color propio, su etiqueta, el contorno y las paredes internas, y un círculo rojo en cada uno. Al tocar un círculo se abre el `AlertDialog` con el título y la descripción. Todo está todavía escrito en el código; eso se corrige en el Paso 2.

A continuación se muestran las capturas mostrando la vista general del plano interactivo y el comportamiento del diálogo flotante al seleccionar cada marcador:

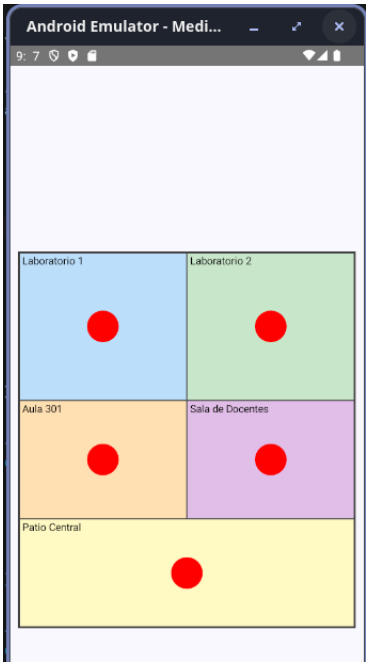


Figura 27: Plano general con 5 ambientes y marcadores visibles



Figura 28: Marcador de Laboratorio 1 presionado



Figura 29: Marcador de Laboratorio 2 presionado



Figura 30: Marcador de Aula 301 presionado



Figura 31: Marcador de Sala de Docentes presionado



Figura 32: Marcador de Patio Central presionado

Paso 2: Convierte la lista de marcadores en datos cargados desde un archivo (por ejemplo un .json o .csv en assets/) en lugar de estar escritos directamente en el código, siguiendo un patrón de persistencia similar al usado en el Laboratorio 03.

Las listas de ambientes y marcadores dejan de estar en el código y se leen de `assets/ambientes.json`. Es el mismo patrón del Laboratorio 03 (datos en un archivo, leídos al abrir la pantalla), pero con lectura de solo lectura desde `assets/` en lugar de `openFileOutput`.

1. Se creo la carpeta assets:
2. Se creo el archivo `ambientes.json` dentro de esa carpeta con el contenido de abajo.
3. Para leer el archivo se utilizaq la función `cargarPlano(context)` abre el archivo con `context.assets.open(...)`, lo convierte a texto y lo interpreta con `org.json.JSONObject`.
4. En el Composable, el `Context` se obtiene con `LocalContext.current` y se carga una sola vez con `remember`.

Archivo `app/src/main/assets/ambientes.json`

MainActivity.kt | ambientes.json

```json
{
  "ambientes": [
    { "etiqueta": "Laboratorio 1",    "x": 10,  "y": 10,  "ancho": 170, "alto": 150, "color": "#BBDEFB" },
    { "etiqueta": "Laboratorio 2",    "x": 180, "y": 10,  "ancho": 170, "alto": 150, "color": "#C8E6C9" },
    { "etiqueta": "Aula 301",         "x": 10,  "y": 160, "ancho": 170, "alto": 120, "color": "#FFE0B2" },
    { "etiqueta": "Sala de Docentes", "x": 180, "y": 160, "ancho": 170, "alto": 120, "color": "#E1BEE7" },
    { "etiqueta": "Patio Central",    "x": 10,  "y": 280, "ancho": 340, "alto": 110, "color": "#FFF9C4" }
  ],
  "marcadores": [
    { "titulo": "Laboratorio 1",    "descripcion": "Laboratorio de cómputo con 30 equipos",        "x": 95,  "y": 85,  "radio": 1
    { "titulo": "Laboratorio 2",    "descripcion": "Laboratorio de redes y servidores",            "x": 265, "y": 85,  "radio": 1
    { "titulo": "Aula 301",         "descripcion": "Aula teórica con capacidad para 40 alumnos",   "x": 95,  "y": 220, "radio": 1
    { "titulo": "Sala de Docentes", "descripcion": "Oficinas y sala de reuniones de docentes",     "x": 265, "y": 220, "radio": 1
    { "titulo": "Patio Central",    "descripcion": "Área abierta de descanso y actividades",       "x": 180, "y": 335, "radio": 1
  ]
}
```

Figura 33: Carpeta assets y archivo ambientes.json en el proyecto

Código implementado en `MainActivity.kt`

```kotlin
fun cargarPlano(context: Context): PlanoData {
    return try {
        val texto = context.assets.open("ambientes.json")
            .bufferedReader()
            .use { it.readText() }
        val raiz = JSONObject(texto)

        val arrayAmbientes = raiz.getJSONArray("ambientes")
        val ambientes = List(arrayAmbientes.length()) { i ->
            val o = arrayAmbientes.getJSONObject(i)
            Ambiente(
                topLeft = Offset(o.getDouble("x").toFloat(), o.getDouble("y").toFloat()),
                size = Size(o.getDouble("ancho").toFloat(), o.getDouble("alto").toFloat()),
                color = Color(android.graphics.Color.parseColor(o.getString("color"))),
                etiqueta = o.getString("etiqueta")
            )
        }

        val arrayMarcadores = raiz.getJSONArray("marcadores")
        val marcadores = List(arrayMarcadores.length()) { i ->
            val o = arrayMarcadores.getJSONObject(i)
            Marcador(
```

```
                offset = Offset(o.getDouble("x").toFloat(), o.getDouble("y").toFloat()),
                radio = o.getDouble("radio").toFloat(),
                titulo = o.getString("titulo"),
                descripcion = o.getString("descripcion")
            )
        }

        PlanoData(ambientes, marcadores)
    } catch (e: Exception) {
        // NO existe el archivo u otro error
        e.printStackTrace()
        PlanoData(emptyList(), emptyList())
    }
}

```

Resultado y Verificación de Persistencia

Para comprobar que la interfaz lee dinámicamente el archivo `ambientes.json` sin depender de datos fijos en el código Kotlin, se realizaron dos pruebas consecutivas:

1. **Lectura inicial (sin cambios):** La aplicación carga la estructura por defecto definida en el archivo de recursos y renderiza el plano correctamente.
2. **Modificación del JSON:** Se editaron los valores en `ambientes.json` (modificando etiquetas, colores de ambiente o descripciones de los marcadores). Al reiniciar la ejecución, la interfaz reflejó automáticamente los nuevos cambios en el emulador.

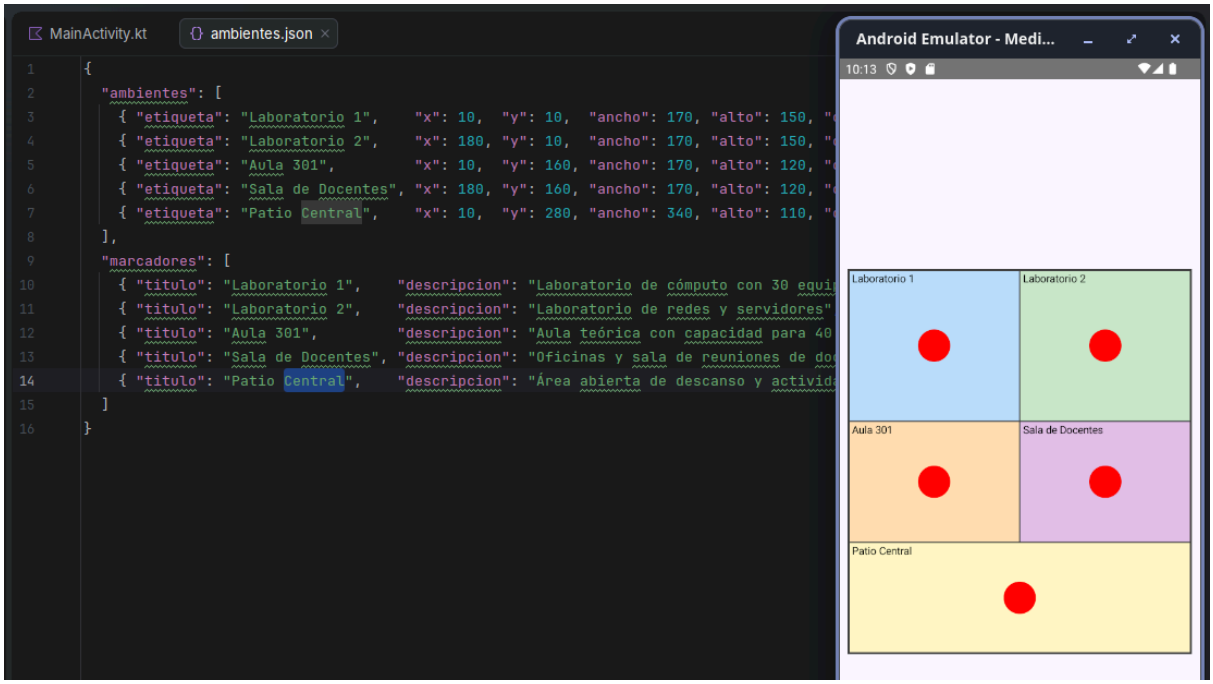


Figura 34: Plano cargado inicialmente desde el archivo JSON

,



Figura 35: Plano tras modificar los datos en ambientes.json

,

Paso 3: . Traslada el estado del plano y sus marcadores a un ViewModel (PlanoViewModel), siguiendo el patrón de ViewModel visto en el Laboratorio 05, en lugar de mantenerlo directamente en el Composable.

La lista de ambientes, la lista de marcadores y el marcador seleccionado dejan de vivir en el Composable y pasan a un `PlanoViewModel`, siguiendo el patrón del Laboratorio 05 (`SeleccionViewModel`).

En `build.gradle.kts` dentro de `dependencies { }`, se agrego y se sincronizo:

```
implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.8.0")
```

- `PlanoViewModel` extiende `AndroidViewModel` porque necesita el `Application` para leer el archivo de `assets/`. Carga el plano una sola vez, en su inicialización.
- `marcadorSeleccionado` es un `mutableStateOf` con `private set`: la interfaz puede leerlo, pero solo el ViewModel puede modificarlo (mediante `onToque` y `cerrarDialogo`).
- `PlanoAmbiente` obtiene el ViewModel con `viewModel()`. Como el estado vive en el ViewModel, sobrevive a rotaciones de pantalla.
- Se retira la `@Preview` porque un `AndroidViewModel` no se puede crear en la vista previa de Android Studio.

Código implementado en `MainActivity.kt`

```kotlin
class PlanoViewModel(application: Application) : AndroidViewModel(application) {

    private val plano = cargarPlano( context = application)

    val ambientes: List<Ambiente> = plano.ambientes
    val marcadores: List<Marcador> = plano.marcadores

    var marcadorSeleccionado by mutableStateOf<Marcador?>( value = null)
        private set

    // Recibe el toque y la escala; decide si cayo dentro de algun marcador
    fun onToque(toque: Offset, escala: Float) {
        marcadores.forEach { marcador ->
            if ((toque - marcador.offset * escala).getDistance() <= marcador.radio * escala) {
                marcadorSeleccionado = marcador
            }
        }
    }

    fun cerrarDialogo() {
        marcadorSeleccionado = null
    }
}
```

Figura 36: Clase `PlanoViewModel` en Android Studio

Resultado

El comportamiento visual es el mismo de los pasos anteriores, pero ahora el Composable no guarda ningún dato: lee todo del ViewModel.

Paso 4: Integra este plano como una nueva pantalla accesible desde MapaScreen (Laboratorio 05) o desde una tarjeta del Laboratorio 06.

Se reproduce la estructura del Laboratorio 05 con las pantallas Home y Mapa, y una tercera ruta `plano` a la que se llega desde un botón de `MapaScreen`. Para realizar esto se necesita dependencias necesarias para la navegación y los íconos extendidos de Compose. Para esto en `build.gradle.kts` (Module :app), agregar y sincronizar:

```
implementation("androidx.navigation:navigation-compose:2.8.0")
implementation("androidx.compose.material:material-icons-extended")
```

- Screen es la sealed class del Laboratorio 05. Se agrega el objeto Plano (ruta «plano»), que no aparece en la barra inferior: solo se llega a él desde MapaScreen.
- MapaScreen recibe la lambda onVerPlano y la ejecuta al pulsar el botón; el NavHost decide qué hacer (navController.navigate(«plano»)), igual que el patrón por lambda del Laboratorio 05.
- PlanoScreen envuelve a PlanoAmbiente con una barra superior sencilla (IconButton con flecha) cuya lambda onVolver llama a navController.popBackStack().
- Mientras se está en el plano, la pestaña **Mapa** permanece resaltada, porque el plano es una sub-pantalla del mapa.
- MainActivity ahora muestra MainScreen() en lugar de PlanoAmbiente() directamente.

Código implementado de `MainActivity.kt`

```

@Composable
fun MainScreen() {
    val navController = rememberNavController()
    val items = listOf(Screen.Home, Screen.Mapa)
    val entradaActual by navController.currentBackStackEntryAsState()
```

```
    val rutaActual = entradaActual?.destination?.route

    Scaffold(
        bottomBar = {
            NavigationBar {
                items.forEach { screen ->
                    val icono = if (screen == Screen.Home) Icons.Default.Home else
Icons.Default.Place
                    NavigationBarItem(
                        selected = rutaActual == screen.route ||
                                (screen == Screen.Mapa && rutaActual == Screen.Plano.route),
                        onClick = { navController.navigate(screen.route) },
                        icon = { Icon(icono, contentDescription = screen.label) },
                        label = { Text(screen.label) }
                    )
                }
            }
        }
    ) { padding ->
        NavHost(
            navController = navController,
            startDestination = Screen.Home.route,
            modifier = Modifier.padding(padding)
        ) {
            composable(Screen.Home.route) {
                HomeScreen()
            }
            composable(Screen.Mapa.route) {
                MapaScreen(onVerPlano = { navController.navigate(Screen.Plano.route) })
            }
            composable(Screen.Plano.route) {
                PlanoScreen(onVolver = { navController.popBackStack() })
            }
        }
    }
}

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            PlanoInteractivo_ComposeTheme {
                MainScreen()
            }
        }
    }
}


```

Resultado

Al abrir la app se ve Home. Desde la pestaña Mapa se pulsa el botón y se abre el plano; la flecha vuelve a Mapa. La barra inferior mantiene el resaltado correcto en todo momento.

```
    implementation("androidx.navigation:navigation-compose:2.8.0")
    implementation("androidx.compose.material:material-icons-extended")
}
```

Figura 37: Dependencias de navegación e íconos agregadas



Figura 38: Pantalla Home

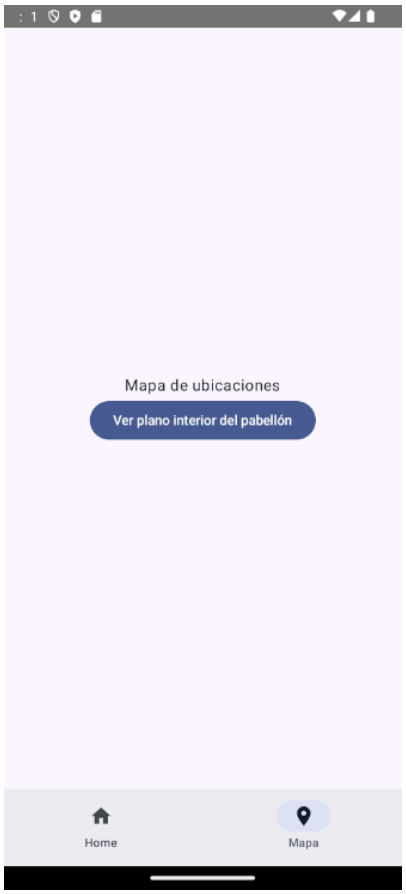


Figura 39: MapaScreen con el botón

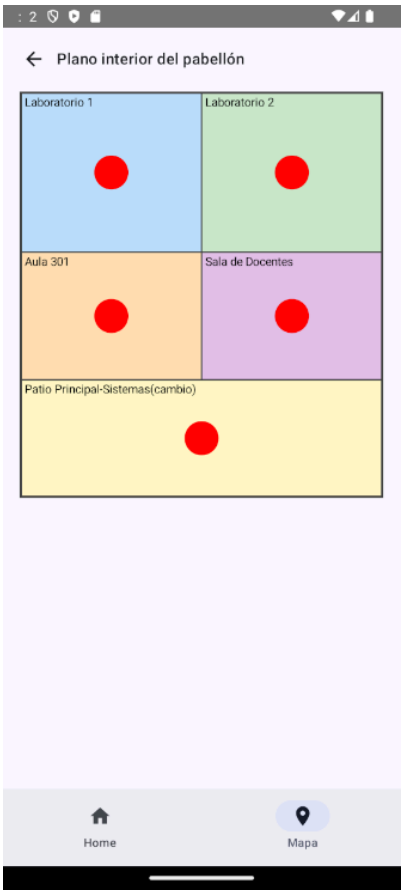


Figura 40: Plano abierto desde Mapa

Paso 5: Verificación final y evidencias del entregable

Se ejecutó la versión final de la aplicación integrando el plano interactivo en el flujo de navegación principal (Pantalla de Inicio → Mapa → Plano de la Edificación). Se comprobó la aplicación de los estilos visuales y la interactividad al seleccionar cada uno de los 5 marcadores, desplegando sus respectivos diálogos modales de información.

Flujo de Navegación

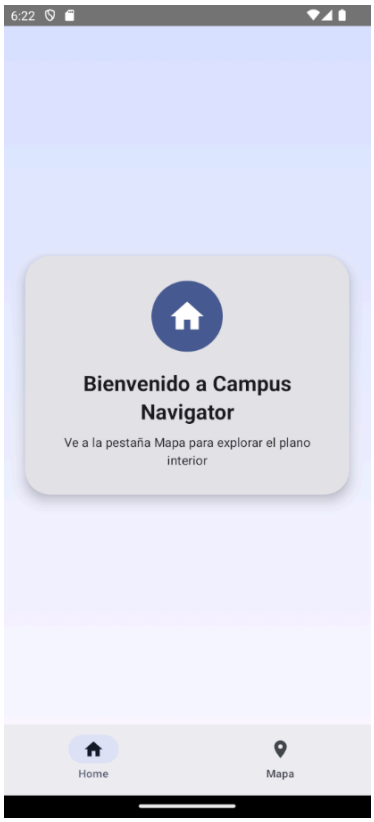


Figura 41: Pantalla de Inicio / Bienvenida



Figura 42: Pantalla del Mapa



Figura 43: Plano Completo con Estilo Aplicado

Interacción con Marcadores (Diálogos Modales)



Figura 44: Laboratorio 1



Figura 45: Laboratorio 2



Figura 46: Aula 301



Figura 47: Sala de Docentes



Figura 48: Patio Central

II. SOLUCIÓN DEL CUESTIONARIO

Pregunta 1: ¿Lograste resolver completamente el Ejercicio/Problema Resuelto por el Docente (plano de un ambiente dibujado con Canvas, donde tocar un marcador abre un diálogo con su información)? Adjunta una captura de pantalla del emulador mostrando el resultado como evidencia.

Respuesta

Sí, logramos resolver de forma completa y satisfactoria el ejercicio propuesto por el docente (`PlanoInteractivo_Compose`). Cada integrante del equipo implementó y verificó el funcionamiento del plano interactivo en su respectivo entorno de desarrollo:

1. **Construcción del lienzo:** Se dibujó el plano geométrico dentro de un `Canvas` de Jetpack Compose, representando los ambientes mediante primitivas `drawRect` y la pared medianera con `drawLine`.
2. **Renderizado de marcadores:** Se definieron las posiciones de los marcadores con coordenadas `Offset` y su radio de cobertura, renderizándolos con `drawCircle`.
3. **Detección de gestos táctiles:** Mediante `Modifier.pointerInput(Unit)` y `detectTapGestures`, se calculó la distancia euclidiana entre el toque del usuario y el centro del marcador para verificar el impacto.
4. **Despliegue del diálogo modal:** Al hacer clic dentro del área activa de un marcador, se actualiza el estado observable `marcadorSeleccionado`, desplegando el diálogo `AlertDialog` con el título y la descripción correspondiente.

Evidencia de Ejecución - Estudiante 1

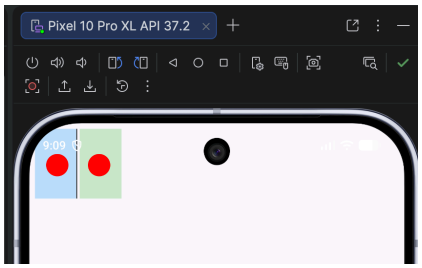


Figura 49: Visualización del plano base en el emulador.



Figura 50: Diálogo informativo modal: Sala A.



Figura 51: Diálogo informativo modal: Sala B.

Evidencia de Ejecución - Estudiante 2

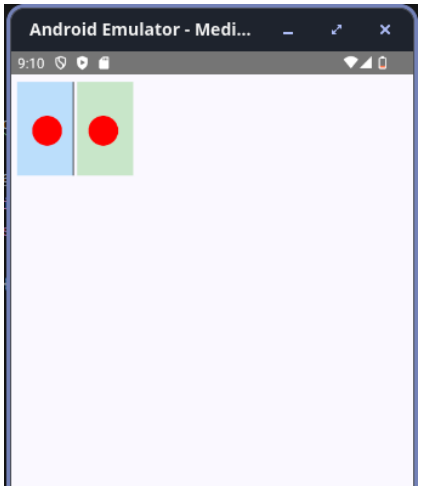


Figura 52: Plano base en emulador (Estudiante 2).



Figura 53: Diálogo modal: Sala A (Estudiante 2).



Figura 54: Diálogo modal: Sala B (Estudiante 2).

Pregunta 2: En tu propia implementación, ¿cómo calculaste si un toque cayó dentro de un marcador circular? ¿Qué otra estrategia se te ocurre para detectar toques sobre formas no circulares, como un rectángulo o un polígono?

1. Cálculo en marcadores circulares

Se utilizó la **distancia euclidiana** entre la coordenada del toque (`tap`) y el centro del marcador (`marcador.offset`):

$$d = \sqrt{(x_t - x_m)^2 + (y_t - y_m)^2} \leq r$$

En Compose se evalúa directamente con `(tap - marcador.offset).getDistance() <= marcador.radio`. Si la distancia es menor o igual al radio, el toque colisiona con el marcador.

2. Estrategias para formas no circulares

- **Rectángulos (Bounding Box):** Se valida si el punto cae dentro de los límites cartesianos ($x_0 \leq x_t \leq x_0 + w$ y $y_0 \leq y_t \leq y_0 + h$), lo cual en Compose se resuelve con `Rect(topLeft, size).contains(tap)`.
- **Polígonos (Ray-Casting):** Se traza un rayo horizontal imaginario desde el toque hacia el infinito; si cruza un número impar de aristas del polígono, el toque está dentro.
- **Regiones nativas (`android.graphics.Region`):** Se define la figura con un `Path` y se delega la comprobación al método nativo `region.contains(x, y)`.

Pregunta 3: En forma individual, comenta en el video de referencia (https://www.youtube.com/watch?v=cCIPYW_N12o) qué otra forma se te ocurre para lograr un dibujo interactivo similar sin depender directamente de Canvas y DrawScope. (Colocar la captura del comentario; uno por integrante del grupo).

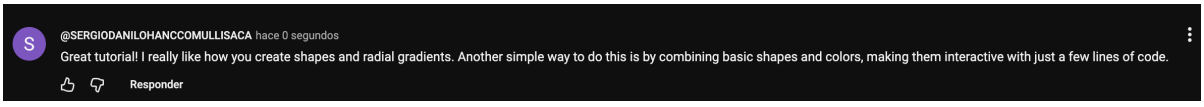


Figura 55: Comentario publicado en el video de referencia de YouTube.

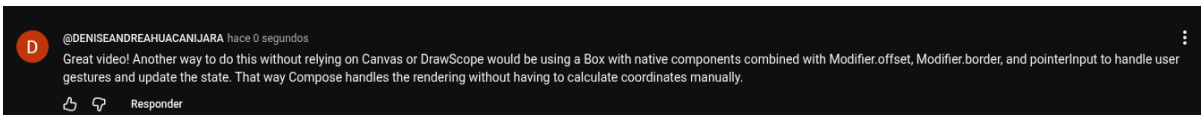


Figura 56: Comentario publicado en el video de referencia de YouTube.

III. CONCLUSIONES

- Redacta aquí la primera conclusión de la práctica.
- Redacta aquí la segunda conclusión de la práctica.
- Redacta aquí la tercera conclusión de la práctica.

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

[1] I. Sommerville, *Ingeniería del Software*, 9.ª ed. Madrid: Pearson Educación, 2011.

[2] R. S. Pressman, *Ingeniería del Software: Un enfoque práctico*, 7.ª ed. México: McGraw-Hill, 2010.

[3] Escuela Profesional de Ingeniería de Sistemas, «Guía de Prácticas de Laboratorio». 2024.